Project Log

RL-ROS2-waffle: PPO navigation training
Branches `first_PPO_training` (Part I) and `PPO_evoution_and_tweaks` (Part II)

Jakub Wilczyński Maksymilian Tulewicz

4–12 November 2026

Part I: first PPO training (`first_PPO_training`)

1 Summary

The branch `first_PPO_training` (commits `6638d6f`, 3 Nov, and `6b1a565`, 4 Nov, on top of `main` at `fcd4d5f`) turns the existing simulation parts (scene generator, map generator, LiDAR, controller) into a complete reinforcement-learning pipeline. The TurtleBot3 Waffle Pi now learns with PPO (Stable-Baselines3) to drive to a goal on randomly generated grid maps in MuJoCo. On the hardest evaluation maps the robot reaches the goal in 20% of episodes at the end of training (best: 40%); most failures are collisions.

2 Changes

Structure and packaging. Modules moved from `src/` to the installable package `src/tb3_nav/`; `pyproject.toml` replaces `requirements.txt` and adds Stable-Baselines3, TensorBoard and Matplotlib. `pip install -e .` provides the commands `tb3-train`, `tb3-play`, `tb3-plot`, `tb3-show`, `tb3-layouts` and `tb3-make-scene`.

Robot model and LiDAR. The wheel velocity actuator gain was raised from `kv=0.1` to `kv=1.0`, so the wheels follow the commanded speed. A LiDAR ray that hits nothing now returns the maximum range (1.5 m) instead of -1, so normalised readings lie in $[0, 1]$.

Gymnasium environment (`nav_env.py`). At every reset a new map is generated on the 10×10 grid (0.5 m cells) and the robot starts at a random cell and heading. The action is applied for 25 physics steps (0.05 s, 20 Hz control). The distance and angle to the goal in the robot frame come from the new `geometry.py`.

Action (2)	$[v,\ \omega] \in [-1, 1]^2$, scaled to 0.26 m/s and 1.82 rad/s
Observation (41)	36 LiDAR readings, robot v and ω, distance to goal, sin/cos of the angle to goal (all normalised)
Reward	10·progress to the goal [m] $-$ 0.01 per step; +15 goal; $-$15 collision
Episode end	goal $\leq$ 0.2 m; collision: LiDAR $\leq$ 0.2 m; limit 500 steps (1500 in the last stage)

Table 1: Definition of `NavEnv`.

PPO training with curriculum (`train_ppo.py`). PPO with 4 parallel environments, $\gamma = 0.995$ (the goal is often 200–400 steps away) and 256×256 policy and value networks. Training moves to the next stage when the robot reaches the goal in 70% of the last 100 episodes. Evaluation always runs on the hardest stage. Every run is saved in its own folder `runs/<date_time>/` with the config, best and final model, TensorBoard logs and plots.

Stage	Map types	Obstacles	Min. start–goal distance	Step limit
0	random	0–3	2 cells	500
1	random	3–8	3 cells	500
2	random	8–15	5 cells	500
3	random, rooms, maze	8–15	5 cells	1500

Table 2: Curriculum stages.

Tools, tests, documentation. `plot_run.py` draws the training curves from the TensorBoard logs; `play_model.py` shows a trained model driving in the MuJoCo viewer and counts goals, collisions and timeouts. New tests cover the goal geometry and the environment (`check_env` for every stage, observation bounds, curriculum switching, goal bonus). The README now describes installation, all commands and the environment.

3 First training results

Two runs are committed: a 200k-step test run (stayed on stage 0) and a 2M-step run `ppo_20261103_225610` (Fig. 1).

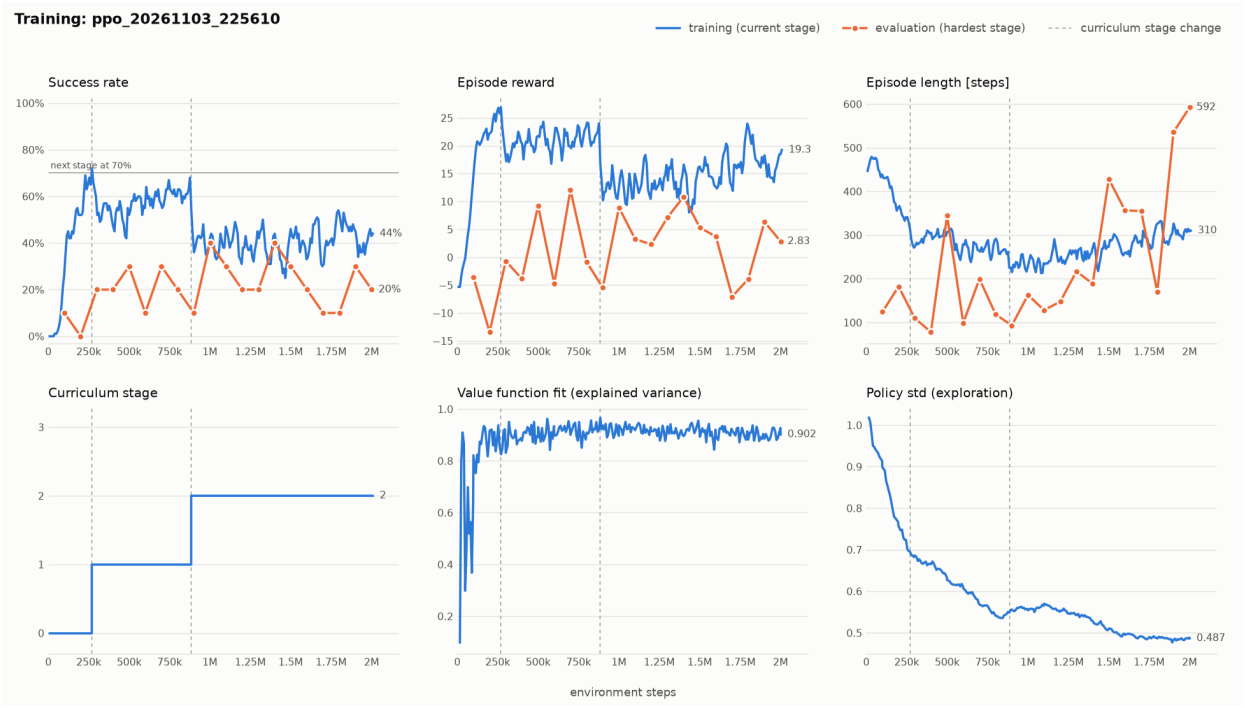


Figure 1: 2M-step run. Blue: training on the current stage; orange: evaluation on the hardest stage (10 episodes each); dashed lines: stage changes.

- **Curriculum:** stage 0 → 1 at about 270k steps and 1 → 2 at about 885k steps. On stage 2 the success rate stayed at about 35–55%, so the last stage (rooms and maze) was never reached.

- **Evaluation:** of all 200 evaluation episodes, 21% reached the goal, 73% ended in a collision and 6% in a timeout.
- **Learning:** the value function fits well (explained variance ≈ 0.9) and the policy becomes more deterministic (std $1.0 \rightarrow 0.49$).

4 Open issues and next steps

1. **Collisions.** The 0.2 m threshold leaves only 5–7 cm around the robot body. Next: a small penalty for getting close to obstacles.
2. **Plateau on stage 2.** Next: longer training or an intermediate stage between 3–8 and 8–15 obstacles.
3. **Noisy evaluation.** 10 episodes per evaluation, and the best model is chosen by mean reward. Next: 50+ episodes on a fixed map set and selection by success rate.
4. **No memory.** The policy sees only the current scan, which is likely to limit it in mazes. Next: frame stacking or a recurrent policy, then ROS 2 integration of the trained policy.

Part II: reward, memory and curriculum (`PPO_evoution_and_tweaks`)

5 Summary

The branch `PPO_evoution_and_tweaks` (10–12 Nov, commits `7dfa4c1`–`7cad22a`) addresses the collisions and the plateau from Part I. Every change was tested on its own (one change per run) and compared on a fixed set of maps. The main outcomes:

- Collisions are now detected by physical contact, and a proximity penalty keeps the robot away from walls. Collisions in rooms dropped from 59% to about 30%.
- Diagnosis: the robot almost always succeeds when the goal is visible, and fails mostly when the goal is *behind a wall*. Reward changes based on straight-line distance did not fix this.
- The decisive change was a **path-distance progress reward** (distance along free cells, used only in the reward). Rooms success rose from 21% (Part I model) to 42%, and random maps from 70% to 77%.
- A standard benchmark (`tb3-bench`) now evaluates every model on the same 600 maps in about 2 minutes.
- An overnight run with a wider maze (1 m corridors) is in progress.

6 Changes

1. Contact-based collisions and proximity penalty (`7dfa4c1`). A collision is now a MuJoCo contact between any robot geom and a wall or block, checked in every physics step, instead of a LiDAR reading ≤ 0.2 m. A linear proximity penalty (0 at 0.25 m, -0.1 per step at 0.15 m or closer) teaches the robot to keep a margin. The physics step was changed to 10×0.005 s (still 0.05 s per action), goal and collision rewards to $+30$ and -50. A new `EpisodeStatsCallback` logs the collision rate, timeout rate and closest obstacle distance to TensorBoard. *Tests:* touching a block ends the episode as a collision, the start pose is not a collision, being close without contact is penalised but not a collision. *Effect:* the robot stopped driving into walls; it sometimes waited in front of narrow gaps instead (timeouts).

2. Record-based progress and a rooms stage (`0834fdf`). Progress is rewarded only when the robot beats its best distance so far, $10 \cdot \max(0,\ d_{\text{best}} - d)$, so moving away and back is not rewarded twice. A curriculum stage with rooms was added before the maze stage, and the evaluation runs on that stage. 8 parallel environments. *Effect:* faster curriculum, but no better final result. A fixed-map diagnosis showed that the robot reaches the goal in ≈98% of episodes when the straight line to the goal is free, but only in ≈60% when it is blocked; all collisions and timeouts happened in the blocked case.

3. Reward experiments that did not help (`4d3da88`, `c5f05c4`). Without the proximity penalty the timeout rate on stage 2 rose from 33% to 50%, so the penalty was kept. A larger time penalty (-0.03 instead of -0.01) lowered random-map success at 2M steps from 75% to 59%, so it was reverted.

4. Memory features in the observation (`6a3e741`, `534ad1e`). Two numbers derived from the robot's own history were added (observation 41 → 43): `gap_from_best`, how much farther from the goal the robot is than its best distance, and `stall`, the time since the last new best distance (normalised to 10 s). Neither uses map information. *Effect:* small and within seed noise; the best checkpoint reached 31% in rooms (Tab. 5).

5. Standard benchmark (`4f88e04`). `tb3-bench <model>` drives the model deterministically on 200 fixed maps of each type (random, rooms, maze; map seeds 20000+) in 8 processes, about 2 minutes per model. It saves per-episode data, a summary, an outcome plot, example trajectories and a comparison of all tested models in `tests/benchmark/`. It also reports success separately for maps where the goal is visible from the start and where it is hidden behind walls.

6. Fixed curriculum schedule (`8c327cc`). The threshold rule (70% success over 100 episodes) depended on luck: the training success on stage 2 is typically 40–55%, so the threshold was only crossed by a lucky streak. Two seeds with the same settings entered the maze stage at 4.6M and 11.0M steps. Stages now change at fixed steps (Tab. 4), so all runs are comparable. The maze stage was removed for now. *Effect:* on its own no improvement (rooms 22% at 8M), but rooms timeouts rose to 51%: the robot did not find the doors.

7. Path-distance progress reward (`7cad22a`). The straight-line distance misleads the robot when the goal is in another room: to get there it must first move away from the goal, towards a door. The progress reward now uses the distance *along free cells*: a breadth-first search from the goal cell over the grid gives the number of cells to the goal, times 0.5 m (inside the goal cell the straight-line distance is used). The map is used **only in the reward, during training in simulation**; the policy still gets the same 43 inputs and no map. This is the usual setup in embodied navigation benchmarks (e.g. the geodesic-distance reward in Habitat PointNav). The memory features still use the straight-line distance, so they carry no map information.

8. Wider maze. In the original maze the corridors are one cell (0.5 m) wide, about 10 cm on each side of the robot; the models collided in 69–83% of maze episodes. `wide_maze()` generates a 5×5 maze and scales it up twice, giving 1 m corridors and walls. The benchmark maze maps now use this generator.

Observation (43)	36 LiDAR readings, v, ω, distance to goal, sin/cos of the angle to goal, `gap_from_best`, `stall`
Reward per step	$10 \cdot \max(0,\ p_{\text{best}} - p)$ with p the path distance to the goal [m]; -0.01; proximity penalty $0 \ldots -0.1$ below 0.25 m
Episode end	goal ≤ 0.2 m: $+30$; contact with a wall or block: -50; step limit (no penalty, truncation)

Table 3: `NavEnv` at the end of Part II (changes from Part I).

Stage	Map types	Obstacles	Min. distance	Step limit	Starts at
0	random	0–3	2 cells	500	0
1	random	3–8	3 cells	500	0.5M
2	random	8–15	5 cells	500	1M
3	random, rooms	8–15	5 cells	1000	2.5M
4	random, rooms, wide maze	8–15	5 cells	1500	16M (overnight run)

Table 4: Curriculum with a fixed schedule.

7 Results

Intermediate experiments. Before the benchmark existed, models were compared on 100 fixed maps per type (the first 100 maps of the benchmark, original narrow maze).

Run	Change	Random maps [%]	Rooms [%]
`ppo_20261110_220127`	record progress, rooms stage	71 / 63 (3M / 5M)	20 / 16
`ppo_20261111_202451`	+ `gap_from_best`	70 / 63 (3M / 5M)	25 / 22
`stall_seed0`	+ `stall`	70 / 73 / 68 (3M / 5M / 12M)	31 / 27 / 21
`stall_seed1`	+ `stall`, other seed	75 / 73 / 72 (3M / 5M / 12M)	24 / 22 / 20

Table 5: Goal reached on fixed maps (deterministic policy).

Benchmark. Table 6 and Fig. 2 compare the final Part I model with the main models of Part II on the same 600 maps. The maze column uses the new wide maze, on which none of these models was trained.

	Random maps		Rooms		
Model	goal	goal hidden	goal	goal hidden	collision / timeout
Part I final (`ppo_20261103_225610`, 2M)	70	62	21	14	59 / 20
memory features (`stall_seed0`, 3M)	73	65	30	23	30 / 40
fixed schedule (`fixed_s0`, 8M)	69	59	22	14	26 / 51
path reward (`pathreward_s0`, 16M)	**77**	**71**	**42**	**37**	35 / 24

Table 6: Benchmark, 200 maps per type, in %. "Goal hidden": the goal is not visible from the start (blocked straight line).

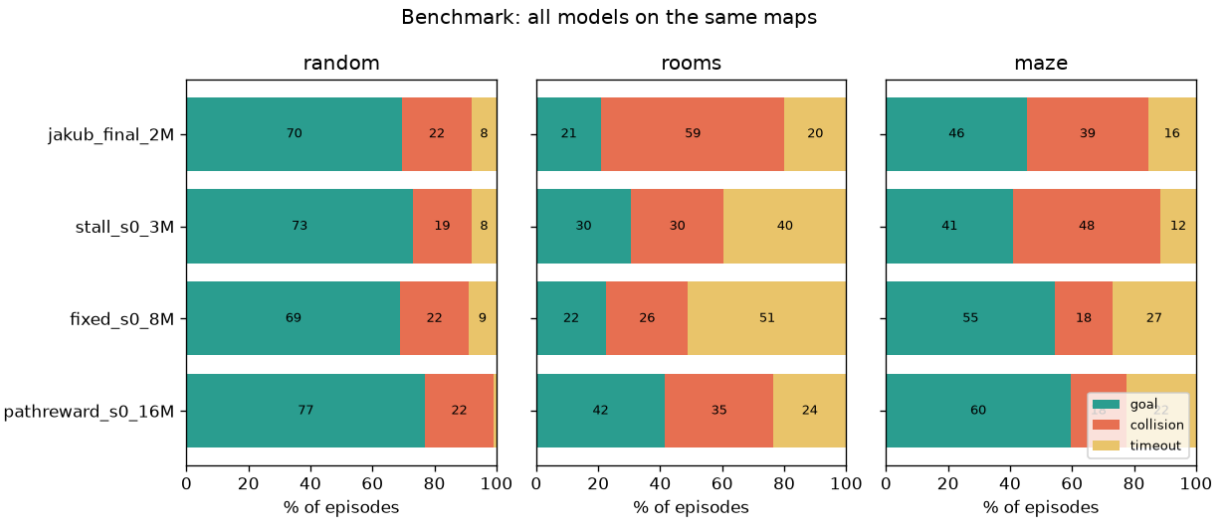


Figure 2: Benchmark outcomes of all models (green: goal, red: collision, yellow: timeout).

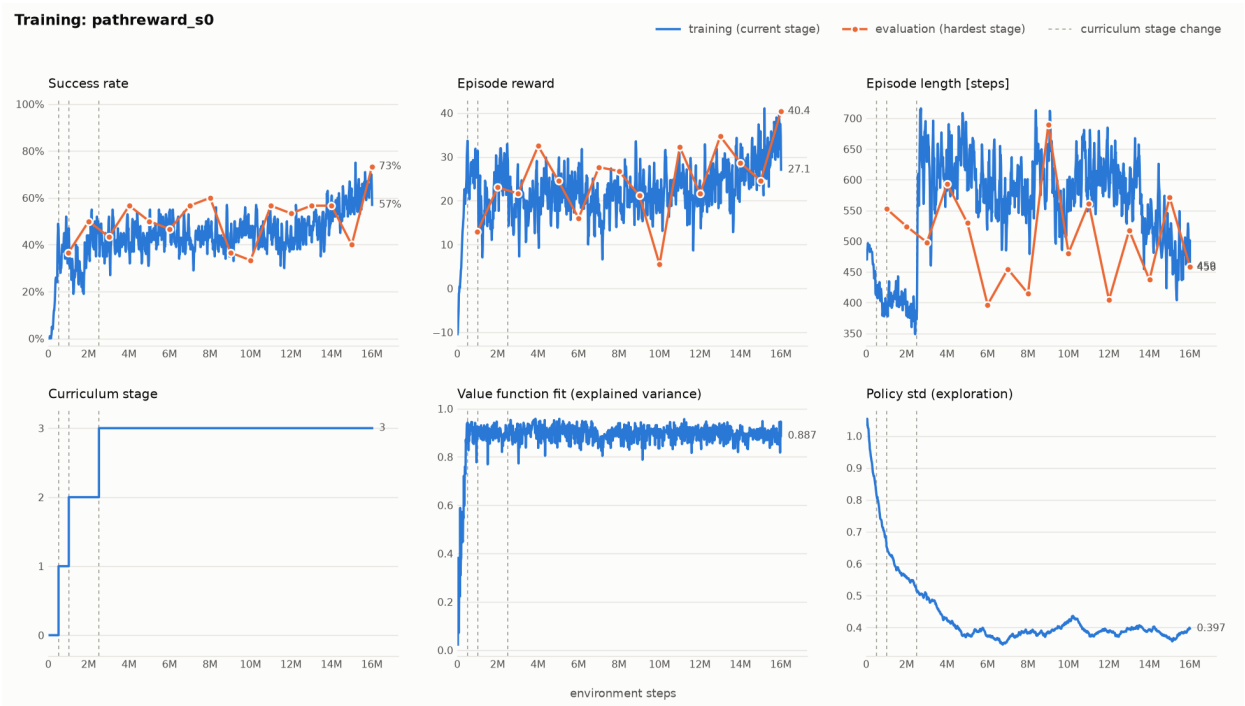


Figure 3: Run `pathreward_s0` (16M steps). The training success rate stays at 40–50% until about 14M steps and then rises to about 60%; the run ended while it was still improving.

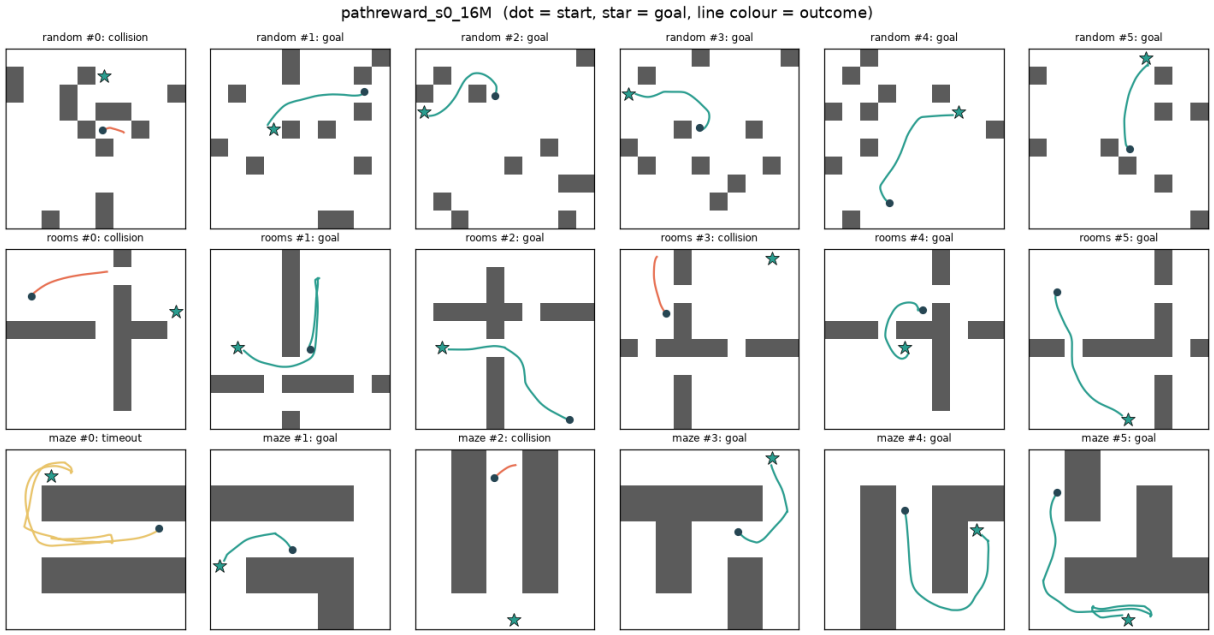


Figure 4: Example paths of `pathreward_s0` on the first six benchmark maps of each type.

Observations.

- **Seed noise** between two runs with the same settings is 5–9 percentage points, so a change needs two seeds or a large effect. The path reward (+11–21 points in rooms) is the first change clearly above this level.
- **Training success is lower than test success** (about 50% vs about 75% on random maps), because training samples random actions and the test uses the mean action.
- **`best_model.zip` is unreliable**: it is chosen by the mean reward of 30 episodes. On 100 fixed

random maps the Part I best model reaches 54%, its final model 74%. Models are now compared with the benchmark only.

- With the straight-line reward, longer training did not help in rooms (results fell from about 30% at 3M to about 20% at 12M).

8 Run in progress and next steps

Overnight run `pathreward_maze_s0`. Same settings as `pathreward_s0` (path reward, fixed schedule, seed 0) for the first 16M steps, then the wide maze is added (stage 4), 30M steps in total. The 16M checkpoint gives a second measurement of the current setup; the rest shows the effect of the maze stage and of longer training. Without any maze training, `pathreward_s0` already reaches the goal in 60% of the wide maze maps (18% collisions).

1. Benchmark the overnight checkpoints (16M, 20M, 25M, 30M).
2. Repeat the best setup with a second seed.
3. Rooms remain the bottleneck (35% collisions, 24% timeouts). Next candidates, one at a time: a recurrent policy (LSTM) for longer memory, more LiDAR rays or range.
4. Export the trained policy to ROS 2.